GROOVEFORGE RECORDS

Smart Vinyl Record Pressing Plant & Inventory System

Complete student implementation manual
Java Swing + JDBC + Oracle

Project scope	Delivery
14 business entities	101 case-study attributes
All-entity CRUD	10 tracking reports
Database design	SQL scripts, constraints and sequences
Student evidence	Expected outputs + labeled local checks

Prepared from the supplied GrooveForge case study, using the hospital manual as a structural reference. No hospital data or hospital modules are carried into this project.

Tracking only: quantities, capacity and status are recorded manually. This system does not calculate prices, automatically adjust stock, or infer production outcomes.

Version 1.0 | 29 September 2026

Evidence boundary: Java compilation and portable database/UI checks were executed locally. Oracle installation, authentication and Oracle-specific acceptance scripts still require a lab run. Illustrations identify the test environment used.

How to use this manual

Part	What to complete
1. Scope and design	Understand the business rules, naming map and relationships.
2. Setup	Create the Oracle schema, build tables, seed the demonstration data.
3. Entity reference	Review all 14 ER attribute sheets, field dictionaries, DDL and examples.
4. Java implementation	Build and run the supplied complete Swing application.
5. Demonstration and outputs	Follow CRUD scenarios and check report outputs.
6. Verification and viva	Run acceptance checks, capture lab evidence and explain the design.

The package contains the complete executable SQL and Java source. Code in this manual explains the important steps; use the actual files when building, rather than copying wrapped lines from the PDF.

Page guide: scope 3-6; setup 7-9; entity reference 10-37; Java 38-41; demonstrations and reports 42-48; tests 49-51; viva 52-53; package map 54.

Output labels

Label	Meaning
Expected Oracle output	A checkpoint to compare after executing on Oracle; not an observed local Oracle result.
Locally verified	Executed using the named local compiler, SQLite-equivalent model or H2 test fixture.
UI illustration	Rendered from the supplied Swing code using H2 sample data; not an Oracle screenshot.

Start-to-finish checklist

Read scope -> verify lab stack -> create project user -> run build.sql -> run acceptance tests -> compile Java -> connect -> exercise each module -> run reports -> capture actual Oracle evidence.

Keep the original case study and hospital reference separate. The case study defines the domain; the reference supplies the guide format and Java/Oracle approach. A current institutional marking rubric was not provided.

1. Project overview and architecture

GrooveForge Records manufactures vinyl releases for independent artists and labels. The application stores equipment details, maintenance history, material receipts, artists, releases, master discs, production batches, material usage, quality inspections, serialized finished records, customer orders, locations and staff.

Layer	Responsibility
Swing screens	Oracle sign-in, entity selector, editable forms, result tables and reports.
JDBC application code	Validate input, bind values, execute SQL, commit success and roll back failures.
Oracle project schema	Persist rows and enforce primary keys, foreign keys, unique values and checks.

```
Student / operator
        |
Java Swing client (grooveforge.App)
        | PreparedStatement + ResultSet
Oracle JDBC driver (lab-approved JAR)
        |
GROOVEFORGE schema: 14 tables + 14 sequences + constraints
```

Learning outcomes

Translate a case study into a relational model; read Chen-style ER symbols; implement DDL and DML; understand cyclic references and composite foreign keys; use JDBC safely; explain transaction boundaries; verify positive and negative cases.

Application access

The login screen authenticates the Oracle schema account through JDBC. It does not add a separate LOGIN business table. Every connected project-schema user has the privileges granted to that schema. EMPLOYEE.role is descriptive and is not an authorization control.

1.1 Scope decisions and exact source mapping

Source name	Implemented name	Reason
RELEASE	VINYL_RELEASE	Clear, portable SQL identifier.
ORDER	CUSTOMER_ORDER	ORDER is SQL syntax.
MASTER_DISC.condition	disc_condition	Explicit meaning and reduced ambiguity.
WAREHOUSE_LOCATION.row	row_label	Avoid the SQL ROW keyword.

Nullability beyond the explicitly optional selected master, validation ranges, uniqueness and date-order checks are implementation choices; the source does not specify a complete constraint policy.

All other source attributes remain present. The count is 101 across the 14 business entities. Constraints and lookup indexes add rules, not extra business attributes.

Design choice	How it is handled
Release/master circular link	Insert release with no selected master, insert its master disc, then update the release. A composite FK guarantees that the selected master belongs to that release.
Finished record release_id	Retained as supplied. A composite FK requires (batch_id, release_id) to match the production batch.
Usage unit	Retained as supplied and checked against the material unit using a composite FK.
Employee and warehouse references	Names and locations remain text. Their standalone registers are not silently connected through invented FKs.
Quality defects	One classification per visual/audio category. Additional observations go in notes. Multiple structured defects would require a schema extension.
Order scope	One release per order; no order-item, record-allocation or payment table is invented.

1.2 Business rules and normalization

Enforced by Oracle

Every row has a positive numeric primary key. Required attributes are NOT NULL. Child links require existing parents. Serial numbers and warehouse shelf/row/section combinations are unique. Enumerated statuses and categories use CHECK constraints. Quantities and dates satisfy the rules shown in the DDL.

A completed batch requires both start and completion timestamps. Completion cannot precede start. Shipped or delivered orders require a shipped date and tracking number. Warehouse occupancy cannot exceed the entered capacity.

Recorded manually; not automatically enforced across events

Maintenance logs do not update machine status or last-maintenance date. Restocks and usage do not adjust stock. A quality pass does not release a batch. Orders do not change production status. No rule automatically prevents recording a batch against a non-operational machine; operators must review the machine register. Sampling against batch size is reviewed by the operator. These are deliberate tracking-scope limits.

Normalization assessment

The design uses one value per attribute and one row per event, supporting 1NF. The business tables use single-column primary keys, with no partial key dependencies under the stated assumptions. It is not accurate to claim every supplied relation is fully in 3NF: BATCH_MATERIAL_USAGE repeats unit determined by material_id, and FINISHED_RECORD repeats release_id determined by batch_id. Those source-required redundancies are retained and protected with composite foreign keys.

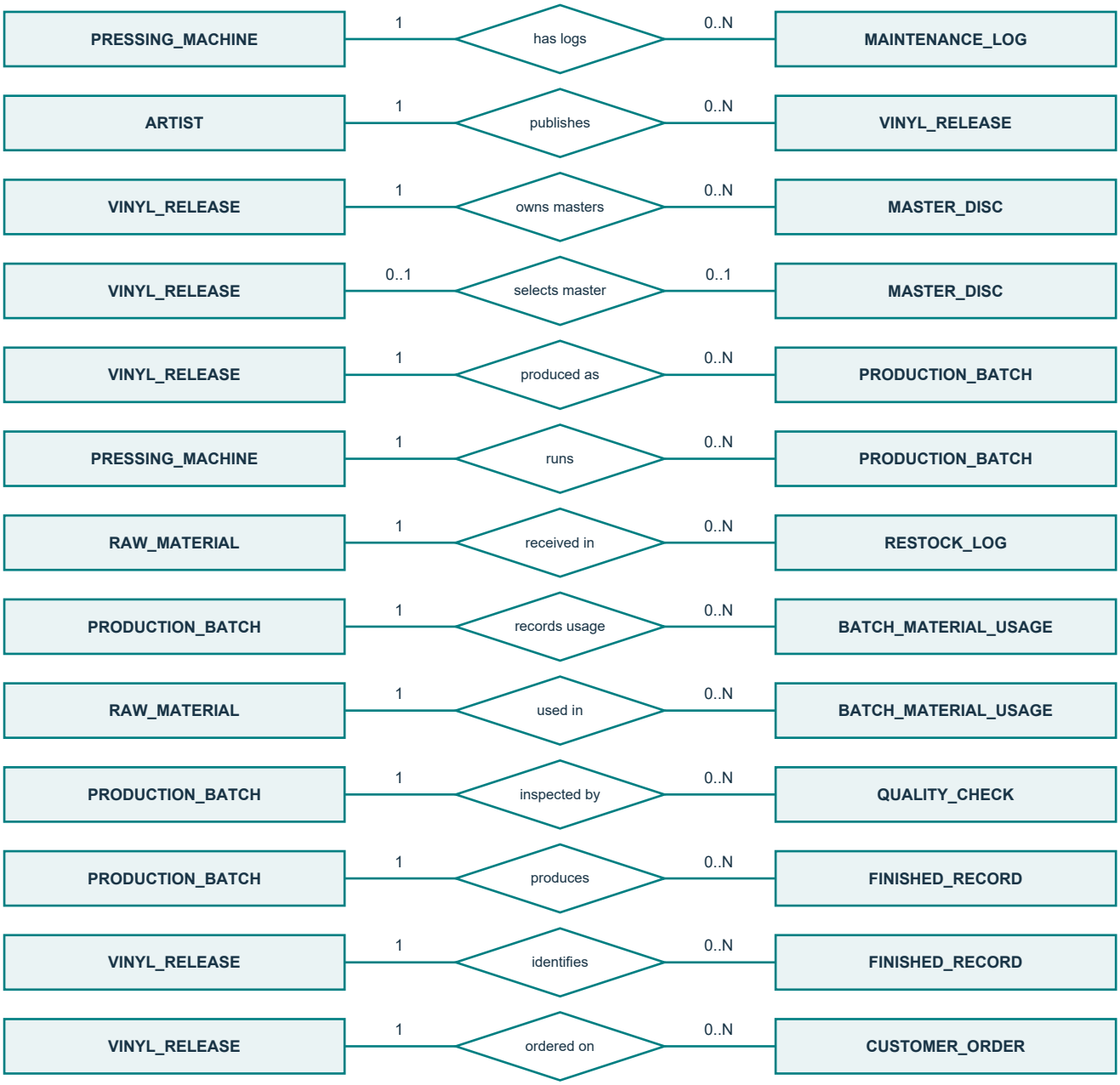
supplier_name in RAW_MATERIAL is the current/default supplier label; supplier_name in RESTOCK_LOG is the historical supplier for that receipt. They may legitimately differ. Staff names are historical text labels rather than stable employee references.

Possible future redesign, outside this delivery

If authorized later, replace text staff/location references with IDs, remove derived duplicate release/unit fields, introduce structured defect details, and implement audited workflow transitions. Those changes would alter the supplied model.

1.3 ER relationship atlas

Each row is a Chen-style relationship. Repeated rectangles refer to the same entity. The entity reference pages supply every attribute oval. This split keeps the 101-attribute model readable.



For the selected-master relationship, each release selects zero or one disc; each disc is selected by zero or one release. Ownership and selection are separate relationships. The composite foreign key restricts selection to the owning release.

Independent entities: EMPLOYEE and WAREHOUSE_LOCATION. Their isolation reflects the supplied case study rather than an omitted relationship.

2. Software verification and Oracle setup

Use the laboratory-approved Oracle, JDK and JDBC driver combination. The hospital reference names Oracle 10g XE, JDK 21 and ojdbc14.jar; this package does not assume that combination is certified. Oracle publishes driver/JDK/database compatibility separately. The Java source uses Java 8 language features and SQL avoids identity columns and other modern-only syntax.

Component	Required action
Oracle	Use the installed lab database; obtain its host, listener port, SID or service name.
JDK	Verify both java and javac are available. A JRE alone can run but cannot compile.
NetBeans	Use a release compatible with the selected JDK. CLI build is also supplied.
JDBC driver	Use the driver approved for that exact Oracle/JDK combination. Put the JAR in lib/.

```
java -version
javac -version

-- In SQL*Plus, after administrator connection:
SELECT * FROM v$version;
SELECT instance_name, status FROM v$instance;
SELECT SYSDATE FROM dual;
```

Expected Oracle checkpoint: an OPEN instance and a successful date query. Version strings depend on the lab. The project user normally does not need access to dynamic performance views.

Connection style	Example only
10g XE SID	jdbc:oracle:thin:@localhost:1521:XE
PDB service	jdbc:oracle:thin:@//localhost:1521/XEPDB1

Newer Oracle installations often use a pluggable database. Create the project user inside the intended PDB. ORA-65096 commonly indicates that a local user was attempted in the CDB root; reconnect to the correct PDB rather than changing hidden Oracle settings.

2.1 Create and build the project schema

1. Open a terminal in the package sql folder. Connect as an administrator to the intended database/PDB. Run the one-time user script and choose a password. The script grants only the project object privileges it needs, with a 50 MB quota on USERS.

```
@00_create_user.sql
```

Expected: User created, Grant succeeded, User altered. If USERS does not exist in the lab, ask the administrator to substitute the appropriate tablespace.

2. Reconnect as GROOVEFORGE using SQL*Plus CONNECT and enter the chosen password when prompted. Do not build tables as SYSTEM.

```
CONNECT grooveforge
SELECT USER FROM dual;
@build.sql
```

Expected: USER is GROOVEFORGE; 14 tables and 14 sequences are created; seed rows are inserted and committed. build_output.txt captures the run. build.sql stops on SQL errors instead of silently continuing.

3. Run the acceptance checks while sample data is unchanged, then examine reports.

```
@05_acceptance_tests.sql
@03_reports.sql
@06_transaction_demo.sql
```

Expected: ALL 14 ACCEPTANCE CHECKS PASSED. The transaction demo shows Quality Hold temporarily, then restores Completed / Ready to Ship using ROLLBACK.

Build order and restart behavior

Parent tables precede children. The optional selected-master constraint is added after both release and master tables exist. The seed inserts releases with NULL selected masters, creates master rows, then assigns selected masters. DDL commits in Oracle, so a failed build can leave partial objects. Use a fresh project schema for a clean retry; no destructive reset is included.

2.2 Seed data and verification checkpoints

Table	Expected seed rows
PRESSING_MACHINE	2
RAW_MATERIAL	3
ARTIST	2
WAREHOUSE_LOCATION	2
EMPLOYEE	3
VINYL_RELEASE	2
MASTER_DISC	2
PRODUCTION_BATCH	3
MAINTENANCE_LOG	2
RESTOCK_LOG	3
BATCH_MATERIAL_USAGE	3
QUALITY_CHECK	2
FINISHED_RECORD	3
CUSTOMER_ORDER	2

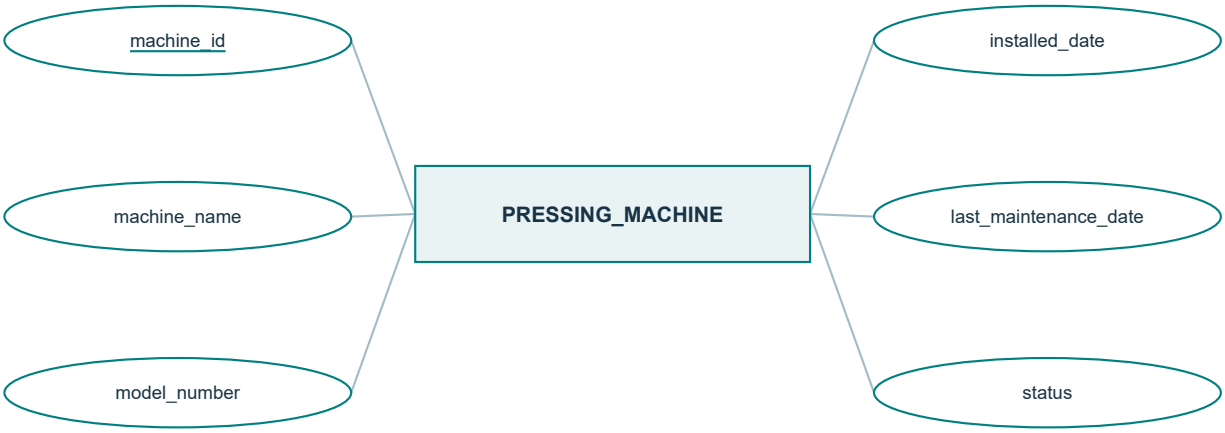
The sample is intentionally compact. It registers three serialized records even though batch 1 has batch_size 100. These are selected tracked examples, not an exhaustive stock ledger. Warehouse occupancy and raw stock are independent manually entered snapshots.

```
SELECT table_name FROM user_tables ORDER BY table_name;
SELECT sequence_name FROM user_sequences ORDER BY sequence_name;
SELECT constraint_name, status FROM user_constraints
WHERE status <> 'ENABLED';
```

Expected: 14 business table names, 14 sequence names, and no disabled constraints. Sequences start at 1000; sample primary keys use smaller values. Failed inserts may consume sequence values, so gaps are normal and IDs should never be promised to be consecutive.

3.1 PRESSING_MACHINE - ER model and fields

Pressing equipment and its manually recorded operating status.



Attribute	Oracle type	Required	Key / allowed values
machine_id	NUMBER(9)	Yes	Primary key; generated by sequence
machine_name	VARCHAR2(80)	Yes	Recorded value
model_number	VARCHAR2(60)	Yes	Recorded value
installed_date	DATE	Yes	Recorded value
last_maintenance_date	DATE	No	Recorded value
status	VARCHAR2(25)	Yes	Operational, Under Maintenance, Faulty, Retired

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.1 PRESSING_MACHINE - implementation

```
CREATE TABLE PRESSING_MACHINE (
  machine_id NUMBER(9) NOT NULL,
  machine_name VARCHAR2(80) NOT NULL,
  model_number VARCHAR2(60) NOT NULL,
  installed_date DATE NOT NULL,
  last_maintenance_date DATE,
  status VARCHAR2(25) NOT NULL,
  PRIMARY KEY (machine_id),
  CHECK (status IN ('Operational', 'Under Maintenance', 'Faulty', 'Retired')),
  CHECK (last_maintenance_date IS NULL OR last_maintenance_date >= installed_date),
  CHECK (machine_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_pressing_machine
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

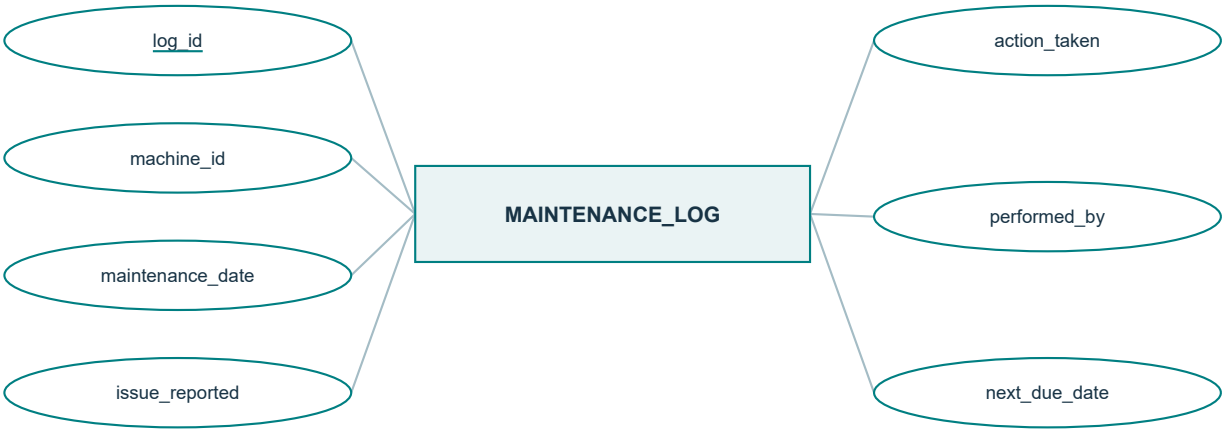
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

machine_id: 1; machine_name: Press Alpha; model_number: GF-120; installed_date: 2024-01-10; last_maintenance_date: 2026-09-01; status: Operational.

3.2 MAINTENANCE_LOG - ER model and fields

One maintenance visit for a machine.



Attribute	Oracle type	Required	Key / allowed values
log_id	NUMBER(9)	Yes	Primary key; generated by sequence
machine_id	NUMBER(9)	Yes	FK -> PRESSING_MACHINE.machine_id
maintenance_date	DATE	Yes	Recorded value
issue_reported	VARCHAR2(250)	Yes	Recorded value
action_taken	VARCHAR2(250)	Yes	Recorded value
performed_by	VARCHAR2(100)	Yes	Recorded value
next_due_date	DATE	No	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.2 MAINTENANCE_LOG - implementation

```
CREATE TABLE MAINTENANCE_LOG (
  log_id NUMBER(9) NOT NULL,
  machine_id NUMBER(9) NOT NULL,
  maintenance_date DATE NOT NULL,
  issue_reported VARCHAR2(250) NOT NULL,
  action_taken VARCHAR2(250) NOT NULL,
  performed_by VARCHAR2(100) NOT NULL,
  next_due_date DATE,
  PRIMARY KEY (log_id),
  FOREIGN KEY (machine_id) REFERENCES PRESSING_MACHINE (machine_id),
  CHECK (next_due_date IS NULL OR next_due_date >= maintenance_date),
  CHECK (log_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_maintenance_log
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

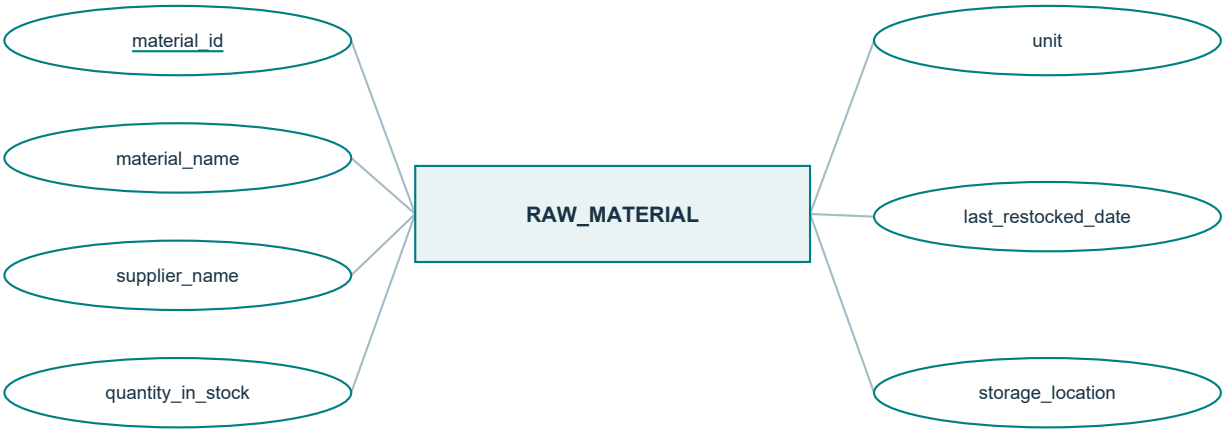
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

log_id: 1; machine_id: 1; maintenance_date: 2026-09-01; issue_reported: Routine service; action_taken: Cleaned and inspected; performed_by: Service Team; next_due_date: 2026-10-01.

3.3 RAW_MATERIAL - ER model and fields

Material catalog and manually recorded stock snapshot.



Attribute	Oracle type	Required	Key / allowed values
material_id	NUMBER(9)	Yes	Primary key; generated by sequence
material_name	VARCHAR2(100)	Yes	Recorded value
supplier_name	VARCHAR2(100)	Yes	Recorded value
quantity_in_stock	NUMBER(12,3)	Yes	Recorded value
unit	VARCHAR2(12)	Yes	Kilograms, Sheets, Rolls, Pieces
last_restocked_date	DATE	No	Recorded value
storage_location	VARCHAR2(100)	Yes	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.3 RAW_MATERIAL - implementation

```
CREATE TABLE RAW_MATERIAL (
  material_id NUMBER(9) NOT NULL,
  material_name VARCHAR2(100) NOT NULL,
  supplier_name VARCHAR2(100) NOT NULL,
  quantity_in_stock NUMBER(12,3) NOT NULL,
  unit VARCHAR2(12) NOT NULL,
  last_restocked_date DATE,
  storage_location VARCHAR2(100) NOT NULL,
  PRIMARY KEY (material_id),
  CHECK (unit IN ('Kilograms', 'Sheets', 'Rolls', 'Pieces')),
  UNIQUE (material_id, unit),
  CHECK (quantity_in_stock >= 0),
  CHECK (material_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_raw_material
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

material_id: 1; material_name: Black Vinyl Pellets; supplier_name: PolyVinyl Supply; quantity_in_stock: 800; unit: Kilograms; last_restocked_date: 2026-09-15; storage_location: Raw Store A.

3.4 RESTOCK_LOG - ER model and fields

Historical receipt; does not automatically increase current stock.



Attribute	Oracle type	Required	Key / allowed values
restock_id	NUMBER(9)	Yes	Primary key; generated by sequence
material_id	NUMBER(9)	Yes	FK -> RAW_MATERIAL.material_id
quantity_added	NUMBER(12,3)	Yes	Recorded value
restocked_date	DATE	Yes	Recorded value
supplier_name	VARCHAR2(100)	Yes	Recorded value
invoice_number	VARCHAR2(60)	Yes	Recorded value
received_by	VARCHAR2(100)	Yes	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.4 RESTOCK_LOG - implementation

```
CREATE TABLE RESTOCK_LOG (
  restock_id NUMBER(9) NOT NULL,
  material_id NUMBER(9) NOT NULL,
  quantity_added NUMBER(12,3) NOT NULL,
  restocked_date DATE NOT NULL,
  supplier_name VARCHAR2(100) NOT NULL,
  invoice_number VARCHAR2(60) NOT NULL,
  received_by VARCHAR2(100) NOT NULL,
  PRIMARY KEY (restock_id),
  FOREIGN KEY (material_id) REFERENCES RAW_MATERIAL (material_id),
  CHECK (quantity_added > 0),
  CHECK (restock_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_restock_log
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

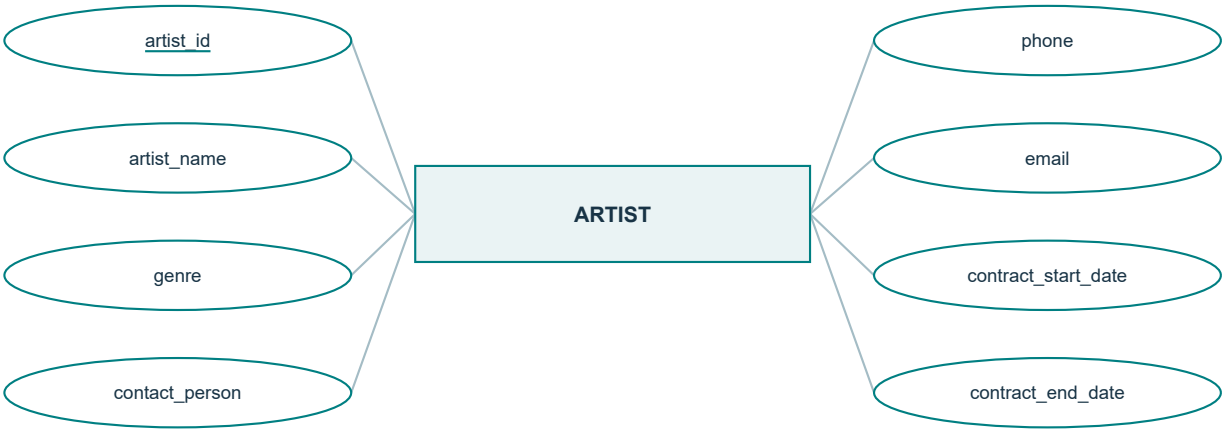
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

restock_id: 1; material_id: 1; quantity_added: 100; restocked_date: 2026-09-15; supplier_name: PolyVinyl Supply; invoice_number: PV-001; received_by: Nisha Patel.

3.5 ARTIST - ER model and fields

Artist or label contact and contract period.



Attribute	Oracle type	Required	Key / allowed values
artist_id	NUMBER(9)	Yes	Primary key; generated by sequence
artist_name	VARCHAR2(100)	Yes	Recorded value
genre	VARCHAR2(60)	Yes	Recorded value
contact_person	VARCHAR2(100)	Yes	Recorded value
phone	VARCHAR2(25)	Yes	Recorded value
email	VARCHAR2(120)	Yes	Recorded value
contract_start_date	DATE	Yes	Recorded value
contract_end_date	DATE	No	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.5 ARTIST - implementation

```
CREATE TABLE ARTIST (
  artist_id NUMBER(9) NOT NULL,
  artist_name VARCHAR2(100) NOT NULL,
  genre VARCHAR2(60) NOT NULL,
  contact_person VARCHAR2(100) NOT NULL,
  phone VARCHAR2(25) NOT NULL,
  email VARCHAR2(120) NOT NULL,
  contract_start_date DATE NOT NULL,
  contract_end_date DATE,
  PRIMARY KEY (artist_id),
  CHECK (contract_end_date IS NULL OR contract_end_date >= contract_start_date),
  CHECK (artist_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_artist
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

artist_id: 1; artist_name: Midnight Atlas; genre: Indie Rock; contact_person: Asha Rao; phone: 9000000001; email: asha@example.com; contract_start_date: 2026-01-01; contract_end_date: 2027-12-31.

3.6 VINYL_RELEASE - ER model and fields

Case-study RELEASE; renamed to avoid ambiguous SQL terminology.



Attribute	Oracle type	Required	Key / allowed values
release_id	NUMBER(9)	Yes	Primary key; generated by sequence
artist_id	NUMBER(9)	Yes	FK -> ARTIST.artist_id
album_title	VARCHAR2(150)	Yes	Recorded value
format	VARCHAR2(10)	Yes	7 Inch, 10 Inch, 12 Inch, LP, EP
color_variant	VARCHAR2(60)	Yes	Recorded value
speed_rpm	NUMBER(2)	Yes	33, 45, 78
total_tracks	NUMBER(3)	Yes	Recorded value
estimated_runtime_minutes	NUMBER(6,2)	Yes	Recorded value
master_vinyl_disc_id	NUMBER(9)	No	FK -> MASTER_DISC.disc_id

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.6 VINYL_RELEASE - implementation

```
CREATE TABLE VINYL_RELEASE (
  release_id NUMBER(9) NOT NULL,
  artist_id NUMBER(9) NOT NULL,
  album_title VARCHAR2(150) NOT NULL,
  format VARCHAR2(10) NOT NULL,
  color_variant VARCHAR2(60) NOT NULL,
  speed_rpm NUMBER(2) NOT NULL,
  total_tracks NUMBER(3) NOT NULL,
  estimated_runtime_minutes NUMBER(6,2) NOT NULL,
  master_vinyl_disc_id NUMBER(9),
  PRIMARY KEY (release_id),
  FOREIGN KEY (artist_id) REFERENCES ARTIST (artist_id),
  CHECK (format IN ('7 Inch', '10 Inch', '12 Inch', 'LP', 'EP')),
  CHECK (speed_rpm IN (33, 45, 78)),
  CHECK (total_tracks > 0),
  CHECK (estimated_runtime_minutes > 0),
  CHECK (release_id > 0)
);
```

After MASTER_DISC exists, add the selected-master constraint:

```
ALTER TABLE VINYL_RELEASE ADD CONSTRAINT fk_selected_master
FOREIGN KEY (master_vinyl_disc_id, release_id)
REFERENCES MASTER_DISC (disc_id, release_id);
```

Sequence and expected result

```
CREATE SEQUENCE seq_vinyl_release
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

release_id: 1; artist_id: 1; album_title: City After Rain; format: LP; color_variant: Black; speed_rpm: 33; total_tracks: 10; estimated_runtime_minutes: 42; master_vinyl_disc_id: 1.

3.7 MASTER_DISC - ER model and fields

Master discs belonging to a release; a release may have several.



Attribute	Oracle type	Required	Key / allowed values
disc_id	NUMBER(9)	Yes	Primary key; generated by sequence
release_id	NUMBER(9)	Yes	FK -> VINYL_RELEASE.release_id
disc_name	VARCHAR2(100)	Yes	Recorded value
created_date	DATE	Yes	Recorded value
created_by	VARCHAR2(100)	Yes	Recorded value
storage_location	VARCHAR2(100)	Yes	Recorded value
disc_condition	VARCHAR2(10)	Yes	New, Good, Worn, Damaged

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.7 MASTER_DISC - implementation

```
CREATE TABLE MASTER_DISC (
  disc_id NUMBER(9) NOT NULL,
  release_id NUMBER(9) NOT NULL,
  disc_name VARCHAR2(100) NOT NULL,
  created_date DATE NOT NULL,
  created_by VARCHAR2(100) NOT NULL,
  storage_location VARCHAR2(100) NOT NULL,
  disc_condition VARCHAR2(10) NOT NULL,
  PRIMARY KEY (disc_id),
  FOREIGN KEY (release_id) REFERENCES VINYL_RELEASE (release_id),
  CHECK (disc_condition IN ('New', 'Good', 'Worn', 'Damaged')),
  UNIQUE (disc_id, release_id),
  CHECK (disc_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_master_disc
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

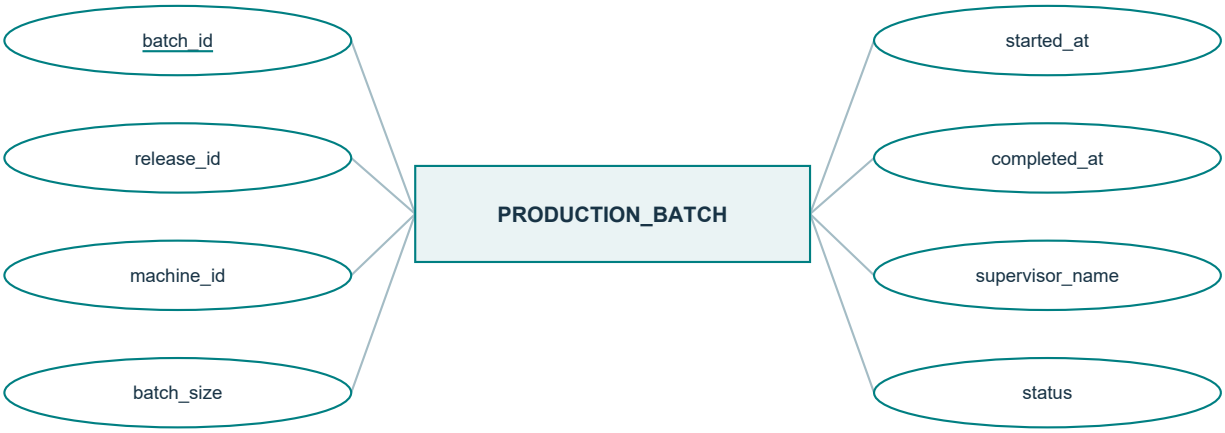
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

disc_id: 1; release_id: 1; disc_name: City Master A; created_date: 2026-09-05; created_by: Riya Sen; storage_location: Master Vault A; disc_condition: Good.

3.8 PRODUCTION_BATCH - ER model and fields

Production run on one machine for one release.



Attribute	Oracle type	Required	Key / allowed values
batch_id	NUMBER(9)	Yes	Primary key; generated by sequence
release_id	NUMBER(9)	Yes	FK -> VINYL_RELEASE.release_id
machine_id	NUMBER(9)	Yes	FK -> PRESSING_MACHINE.machine_id
batch_size	NUMBER(9)	Yes	Recorded value
started_at	TIMESTAMP	No	Recorded value
completed_at	TIMESTAMP	No	Recorded value
supervisor_name	VARCHAR2(100)	Yes	Recorded value
status	VARCHAR2(15)	Yes	Scheduled, In Progress, Completed, Quality Hold, Rejected

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.8 PRODUCTION_BATCH - implementation

```
CREATE TABLE PRODUCTION_BATCH (
  batch_id NUMBER(9) NOT NULL,
  release_id NUMBER(9) NOT NULL,
  machine_id NUMBER(9) NOT NULL,
  batch_size NUMBER(9) NOT NULL,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  supervisor_name VARCHAR2(100) NOT NULL,
  status VARCHAR2(15) NOT NULL,
  PRIMARY KEY (batch_id),
  FOREIGN KEY (release_id) REFERENCES VINYL_RELEASE (release_id),
  FOREIGN KEY (machine_id) REFERENCES PRESSING_MACHINE (machine_id),
  CHECK (status IN ('Scheduled', 'In Progress', 'Completed', 'Quality Hold', 'Rejected')),
  UNIQUE (batch_id, release_id),
  CHECK (batch_size > 0),
  CHECK ((completed_at IS NULL OR (started_at IS NOT NULL AND completed_at >= started_at))),
  CHECK (status NOT IN ('In Progress','Completed') OR started_at IS NOT NULL),
  CHECK (status <> 'Completed' OR completed_at IS NOT NULL),
  CHECK (batch_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_production_batch
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

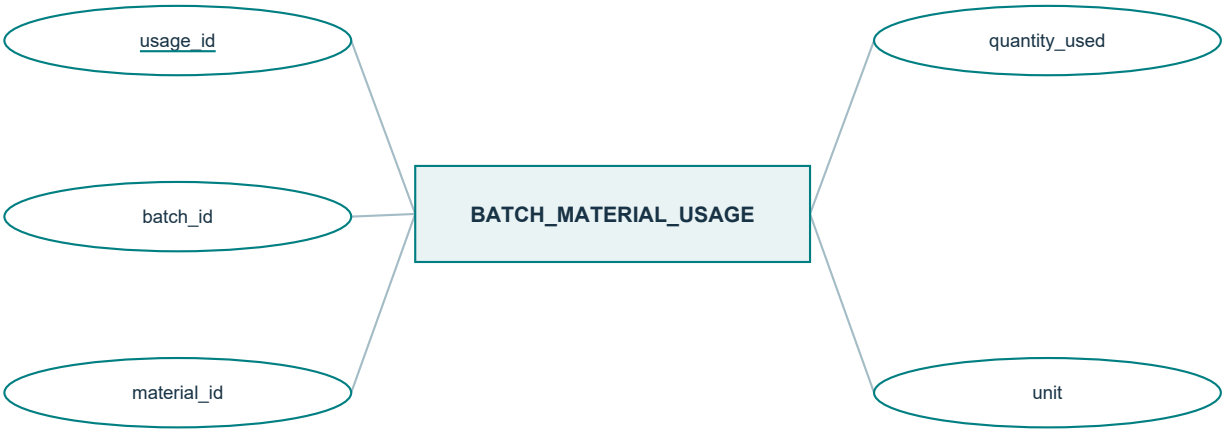
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

batch_id: 1; release_id: 1; machine_id: 1; batch_size: 100; started_at: 2026-09-18 09:00:00; completed_at: 2026-09-18 13:00:00; supervisor_name: Riya Sen; status: Completed.

3.9 BATCH_MATERIAL_USAGE - ER model and fields

Recorded material usage; stock is not automatically deducted.



Attribute	Oracle type	Required	Key / allowed values
usage_id	NUMBER(9)	Yes	Primary key; generated by sequence
batch_id	NUMBER(9)	Yes	FK -> PRODUCTION_BATCH.batch_id
material_id	NUMBER(9)	Yes	FK -> RAW_MATERIAL.material_id
quantity_used	NUMBER(12,3)	Yes	Recorded value
unit	VARCHAR2(12)	Yes	Kilograms, Sheets, Rolls, Pieces

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.9 BATCH_MATERIAL_USAGE - implementation

```
CREATE TABLE BATCH_MATERIAL_USAGE (
  usage_id NUMBER(9) NOT NULL,
  batch_id NUMBER(9) NOT NULL,
  material_id NUMBER(9) NOT NULL,
  quantity_used NUMBER(12,3) NOT NULL,
  unit VARCHAR2(12) NOT NULL,
  PRIMARY KEY (usage_id),
  FOREIGN KEY (batch_id) REFERENCES PRODUCTION_BATCH (batch_id),
  FOREIGN KEY (material_id) REFERENCES RAW_MATERIAL (material_id),
  CHECK (unit IN ('Kilograms', 'Sheets', 'Rolls', 'Pieces')),
  CHECK (quantity_used > 0),
  CHECK (usage_id > 0),
  CONSTRAINT fk_usage_material_unit FOREIGN KEY (material_id, unit) REFERENCES RAW_MATERIAL
    (material_id, unit)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_batch_material_usage
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

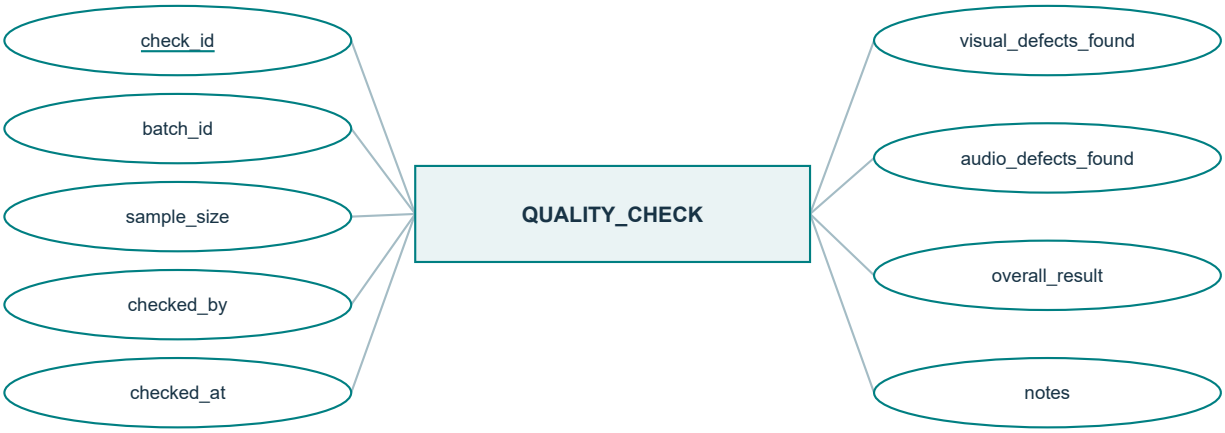
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

usage_id: 1; batch_id: 1; material_id: 1; quantity_used: 18; unit: Kilograms.

3.10 QUALITY_CHECK - ER model and fields

Inspection event; one selected defect classification per category.



Attribute	Oracle type	Required	Key / allowed values
check_id	NUMBER(9)	Yes	Primary key; generated by sequence
batch_id	NUMBER(9)	Yes	FK -> PRODUCTION_BATCH.batch_id
sample_size	NUMBER(9)	Yes	Recorded value
checked_by	VARCHAR2(100)	Yes	Recorded value
checked_at	TIMESTAMP	Yes	Recorded value
visual_defects_found	VARCHAR2(25)	Yes	Scratches, Warping, Bubbles, Color Inconsistency, None
audio_defects_found	VARCHAR2(15)	Yes	Pops, Clicks, Skips, Distortion, None
overall_result	VARCHAR2(20)	Yes	Pass, Conditional Pass, Fail
notes	VARCHAR2(500)	No	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.10 QUALITY_CHECK - implementation

```
CREATE TABLE QUALITY_CHECK (
  check_id NUMBER(9) NOT NULL,
  batch_id NUMBER(9) NOT NULL,
  sample_size NUMBER(9) NOT NULL,
  checked_by VARCHAR2(100) NOT NULL,
  checked_at TIMESTAMP NOT NULL,
  visual_defects_found VARCHAR2(25) NOT NULL,
  audio_defects_found VARCHAR2(15) NOT NULL,
  overall_result VARCHAR2(20) NOT NULL,
  notes VARCHAR2(500),
  PRIMARY KEY (check_id),
  FOREIGN KEY (batch_id) REFERENCES PRODUCTION_BATCH (batch_id),
  CHECK (visual_defects_found IN ('Scratches', 'Warping', 'Bubbles', 'Color Inconsistency',
    'None')),
  CHECK (audio_defects_found IN ('Pops', 'Clicks', 'Skips', 'Distortion', 'None')),
  CHECK (overall_result IN ('Pass', 'Conditional Pass', 'Fail')),
  CHECK (sample_size > 0),
  CHECK (check_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_quality_check
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

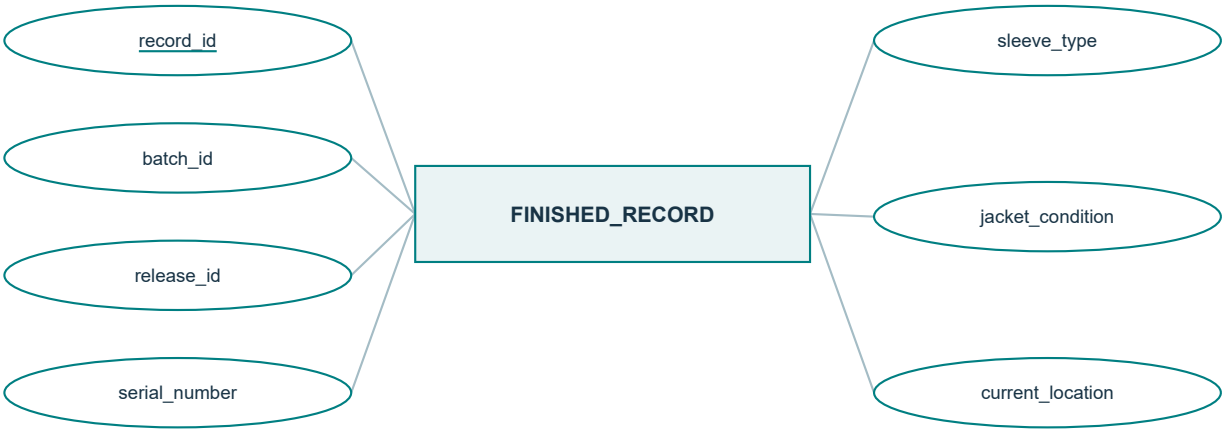
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

check_id: 1; batch_id: 1; sample_size: 10; checked_by: Dev Mehta; checked_at: 2026-09-18 14:00:00; visual_defects_found: None; audio_defects_found: None; overall_result: Pass; notes: Sample approved.

3.11 FINISHED_RECORD - ER model and fields

Individually serialized record; batch and release must agree.



Attribute	Oracle type	Required	Key / allowed values
record_id	NUMBER(9)	Yes	Primary key; generated by sequence
batch_id	NUMBER(9)	Yes	FK -> PRODUCTION_BATCH.batch_id
release_id	NUMBER(9)	Yes	FK -> VINYL_RELEASE.release_id
serial_number	VARCHAR2(60)	Yes	Recorded value
sleeve_type	VARCHAR2(25)	Yes	Standard, Gatefold, Picture Disc Sleeve
jacket_condition	VARCHAR2(12)	Yes	Mint, Near Mint, Good, Fair
current_location	VARCHAR2(100)	Yes	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.11 FINISHED_RECORD - implementation

```
CREATE TABLE FINISHED_RECORD (
  record_id NUMBER(9) NOT NULL,
  batch_id NUMBER(9) NOT NULL,
  release_id NUMBER(9) NOT NULL,
  serial_number VARCHAR2(60) NOT NULL,
  sleeve_type VARCHAR2(25) NOT NULL,
  jacket_condition VARCHAR2(12) NOT NULL,
  current_location VARCHAR2(100) NOT NULL,
  PRIMARY KEY (record_id),
  FOREIGN KEY (batch_id) REFERENCES PRODUCTION_BATCH (batch_id),
  FOREIGN KEY (release_id) REFERENCES VINYL_RELEASE (release_id),
  CHECK (sleeve_type IN ('Standard', 'Gatefold', 'Picture Disc Sleeve')),
  CHECK (jacket_condition IN ('Mint', 'Near Mint', 'Good', 'Fair')),
  UNIQUE (serial_number),
  CHECK (record_id > 0),
  CONSTRAINT fk_record_batch_release FOREIGN KEY (batch_id, release_id) REFERENCES
    PRODUCTION_BATCH (batch_id, release_id)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_finished_record
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

record_id: 1; batch_id: 1; release_id: 1; serial_number: GF-CAR-0001; sleeve_type: Standard; jacket_condition: Mint; current_location: Warehouse Shelf A3.

3.12 CUSTOMER_ORDER - ER model and fields

Case-study ORDER; one release per order.



Attribute	Oracle type	Required	Key / allowed values
order_id	NUMBER(9)	Yes	Primary key; generated by sequence
release_id	NUMBER(9)	Yes	FK -> VINYL_RELEASE.release_id
customer_name	VARCHAR2(100)	Yes	Recorded value
quantity_ordered	NUMBER(9)	Yes	Recorded value
order_date	DATE	Yes	Recorded value
status	VARCHAR2(15)	Yes	Pending, In Production, Quality Hold, Ready to Ship, Shipped, Delivered
shipping_address	VARCHAR2(250)	Yes	Recorded value
shipped_date	DATE	No	Recorded value
tracking_number	VARCHAR2(80)	No	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.12 CUSTOMER_ORDER - implementation

```
CREATE TABLE CUSTOMER_ORDER (
  order_id NUMBER(9) NOT NULL,
  release_id NUMBER(9) NOT NULL,
  customer_name VARCHAR2(100) NOT NULL,
  quantity_ordered NUMBER(9) NOT NULL,
  order_date DATE NOT NULL,
  status VARCHAR2(15) NOT NULL,
  shipping_address VARCHAR2(250) NOT NULL,
  shipped_date DATE,
  tracking_number VARCHAR2(80),
  PRIMARY KEY (order_id),
  FOREIGN KEY (release_id) REFERENCES VINYL_RELEASE (release_id),
  CHECK (status IN ('Pending', 'In Production', 'Quality Hold', 'Ready to Ship', 'Shipped',
    'Delivered')),
  CHECK (quantity_ordered > 0),
  CHECK (shipped_date IS NULL OR shipped_date >= order_date),
  CHECK (status NOT IN ('Shipped','Delivered') OR (shipped_date IS NOT NULL AND tracking_number
    IS NOT NULL)),
  CHECK (order_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_customer_order
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

order_id: 1; release_id: 1; customer_name: Atlas Label; quantity_ordered: 100; order_date: 2026-09-10; status: Ready to Ship; shipping_address: 10 Example Road, Mumbai; shipped_date: NULL; tracking_number: NULL.

3.13 WAREHOUSE_LOCATION - ER model and fields

Independent location register; capacity is entered manually.



Attribute	Oracle type	Required	Key / allowed values
location_id	NUMBER(9)	Yes	Primary key; generated by sequence
shelf_number	VARCHAR2(20)	Yes	Recorded value
row_label	VARCHAR2(20)	Yes	Recorded value
section	VARCHAR2(40)	Yes	Recorded value
current_capacity_used	NUMBER(9)	Yes	Recorded value
max_capacity	NUMBER(9)	Yes	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.13 WAREHOUSE_LOCATION - implementation

```
CREATE TABLE WAREHOUSE_LOCATION (
  location_id NUMBER(9) NOT NULL,
  shelf_number VARCHAR2(20) NOT NULL,
  row_label VARCHAR2(20) NOT NULL,
  section VARCHAR2(40) NOT NULL,
  current_capacity_used NUMBER(9) NOT NULL,
  max_capacity NUMBER(9) NOT NULL,
  PRIMARY KEY (location_id),
  UNIQUE (shelf_number, row_label, section),
  CHECK (current_capacity_used >= 0),
  CHECK (max_capacity > 0),
  CHECK (current_capacity_used <= max_capacity),
  CHECK (location_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_warehouse_location
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

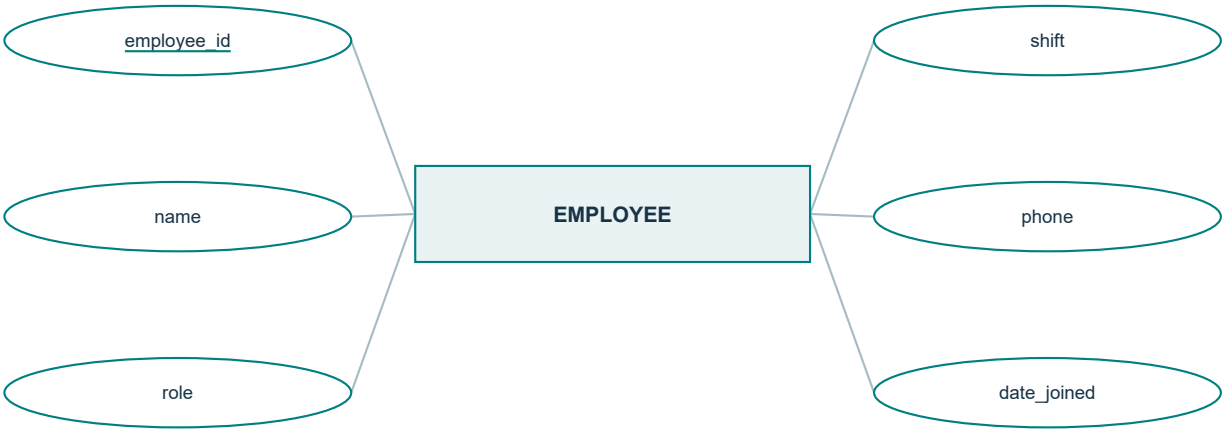
Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

location_id: 1; shelf_number: A3; row_label: A; section: Finished; current_capacity_used: 2; max_capacity: 500.

3.14 EMPLOYEE - ER model and fields

Independent staff directory; role is not an application permission.



Attribute	Oracle type	Required	Key / allowed values
employee_id	NUMBER(9)	Yes	Primary key; generated by sequence
name	VARCHAR2(100)	Yes	Recorded value
role	VARCHAR2(20)	Yes	Press Operator, Quality Inspector, Warehouse Staff, Supervisor, Admin
shift	VARCHAR2(10)	Yes	Morning, Evening, Night
phone	VARCHAR2(25)	Yes	Recorded value
date_joined	DATE	Yes	Recorded value

Underlined oval = primary key. FK fields are included as logical attributes because the supplied case study explicitly lists them. This is a logical ER atlas rather than a pure conceptual diagram that hides foreign-key columns.

3.14 EMPLOYEE - implementation

```
CREATE TABLE EMPLOYEE (
  employee_id NUMBER(9) NOT NULL,
  name VARCHAR2(100) NOT NULL,
  role VARCHAR2(20) NOT NULL,
  shift VARCHAR2(10) NOT NULL,
  phone VARCHAR2(25) NOT NULL,
  date_joined DATE NOT NULL,
  PRIMARY KEY (employee_id),
  CHECK (role IN ('Press Operator', 'Quality Inspector', 'Warehouse Staff', 'Supervisor',
    'Admin')),
  CHECK (shift IN ('Morning', 'Evening', 'Night')),
  CHECK (employee_id > 0)
);
```

Sequence and expected result

```
CREATE SEQUENCE seq_employee
START WITH 1000 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Expected Oracle output: Table created. Sequence created. The global schema script creates all tables in dependency order. Do not execute these pages in chapter order against an empty schema.

First sample record

employee_id: 1; name: Riya Sen; role: Supervisor; shift: Morning; phone: 9000000011; date_joined: 2025-01-10.

4. Build the complete Java application

All source files are supplied. The application creates Swing controls in code, so no NetBeans form designer steps or generated variable names are required.

File	Role
src/grooveforge/App.java	Login, module navigation, forms, validation, JDBC operations and reports.
src/grooveforge/Model.java	Fixed table/column allowlist, enums, parent selectors and report SQL.
src/grooveforge/TestConnection.java	Console connection and table-count checkpoint.
build.cmd / run.cmd	Compile the source and launch with the lib folder on the classpath.

A precompiled GrooveForge.jar is included. run.cmd uses freshly compiled build classes when present, otherwise this JAR. Running requires Java and the Oracle driver; compiling additionally requires a JDK.

Command-line build

```
build.cmd
run.cmd
```

Expected: Build successful, then the Oracle sign-in window. If javac is missing, install or select the lab JDK. Put the compatible JDBC JAR in lib before connecting. Compilation itself needs only the standard JDK because driver classes are loaded by name at runtime.

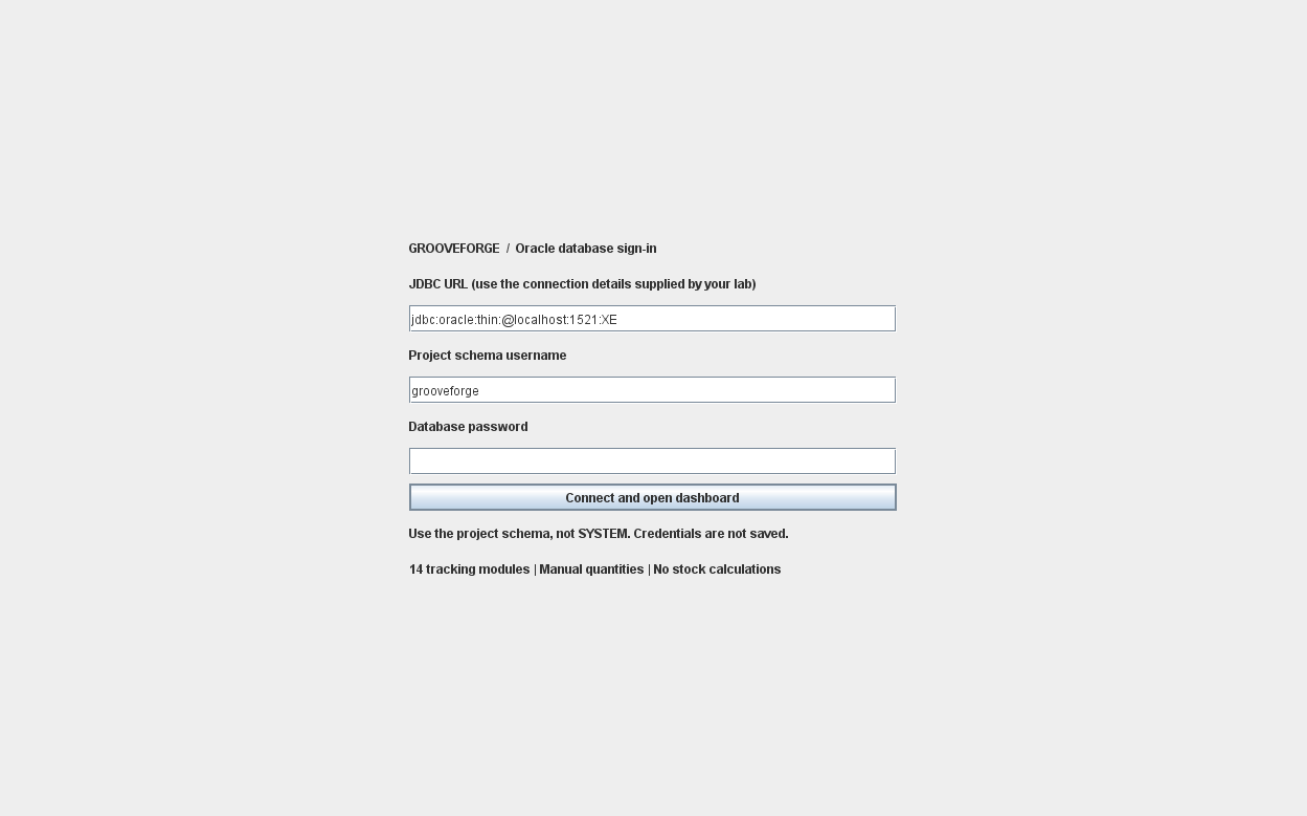
NetBeans workflow

1. Create a Java project with existing sources (Java with Ant, or the equivalent option in the installed version). Select the project folder and src as the source root.
2. Add the Oracle JDBC JAR under Libraries.
3. Set the main class to grooveforge.App and select the lab JDK.
4. Clean and Build; Run Project.
5. Enter the actual JDBC URL, GROOVEFORGE username and chosen password.

The console checkpoint can be run from a terminal with:
`java -cp "build;lib/*" grooveforge.TestConnection`
It uses a console password prompt; in the NetBeans output pane, use the Swing sign-in instead.

4.1 Connection and dashboard output

The application clears the password field after connection is requested. A successful connection checks that all 14 expected tables are accessible before opening the dashboard. A missing driver, invalid credentials or missing schema produces an error dialog.

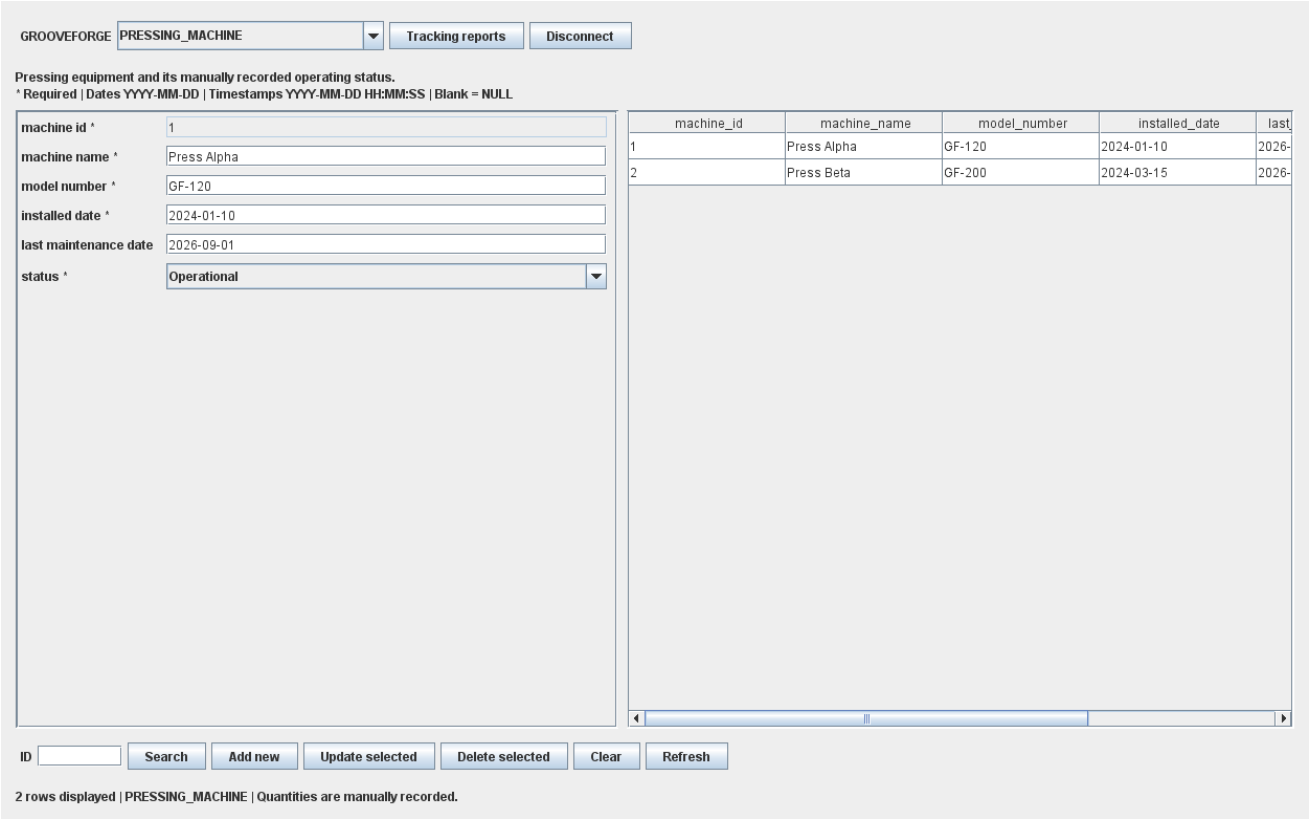


UI illustration: actual Swing component rendering. No Oracle login was executed locally.

Test	Expected behavior
Valid project credentials and built schema	Dashboard opens with machine rows.
Wrong password	Connection is rejected; dashboard stays closed.
Missing JDBC JAR	Driver-missing message.
Missing tables or wrong schema	Project-tables-missing message.
Disconnect	Database connection closes and login screen returns.

4.2 CRUD workflow used by all 14 modules

Choose a module from the top selector. Parent foreign keys use ID-and-label choices. Enumerated values use drop-down controls. Optional fields may be blank; the application writes SQL NULL.



UI illustration: supplied Swing code rendered with H2 sample data, not Oracle.

Action	Procedure / expected behavior
Add new	Clear -> fill required fields -> Add new. A sequence supplies the ID; a success dialog reports it.
Search	Enter an ID -> Search. Matching data fills the form; otherwise ID not found.
Update selected	Select a row -> change values -> Update selected. Exactly one row must be updated.
Delete selected	Select -> Delete -> confirm. Parent rows with child references are rejected.
Clear / Refresh	Clear prepares an empty form. Refresh reloads committed rows and parent choices.

4.3 JDBC implementation and transaction boundary

Each module reuses the same tested form engine. Table and column identifiers come from Model.java, never from a text box. Values are bound using PreparedStatement. Date and number parsing occurs before JDBC writes.

```
SELECT seq_pressing_machine.NEXTVAL FROM dual;

INSERT INTO PRESSING_MACHINE
(machine_name, model_number, installed_date,
 last_maintenance_date, status, machine_id)
VALUES (?, ?, ?, ?, ?, ?);

UPDATE PRESSING_MACHINE
SET machine_name = ?, model_number = ?, installed_date = ?,
    last_maintenance_date = ?, status = ?
WHERE machine_id = ?;

connection.setAutoCommit(false);
try {
    // Bind user values with setString / setBigDecimal / setDate.
    // Execute exactly one insert, update or delete.
    connection.commit();
} catch (Exception ex) {
    connection.rollback();
    throw ex;
}
```

The example is explanatory; App.java contains complete executable methods. A SwingWorker performs database work off the event-dispatch thread. Controls are temporarily disabled to keep the selected module and input stable during the operation. The result table refreshes only after a successful commit.

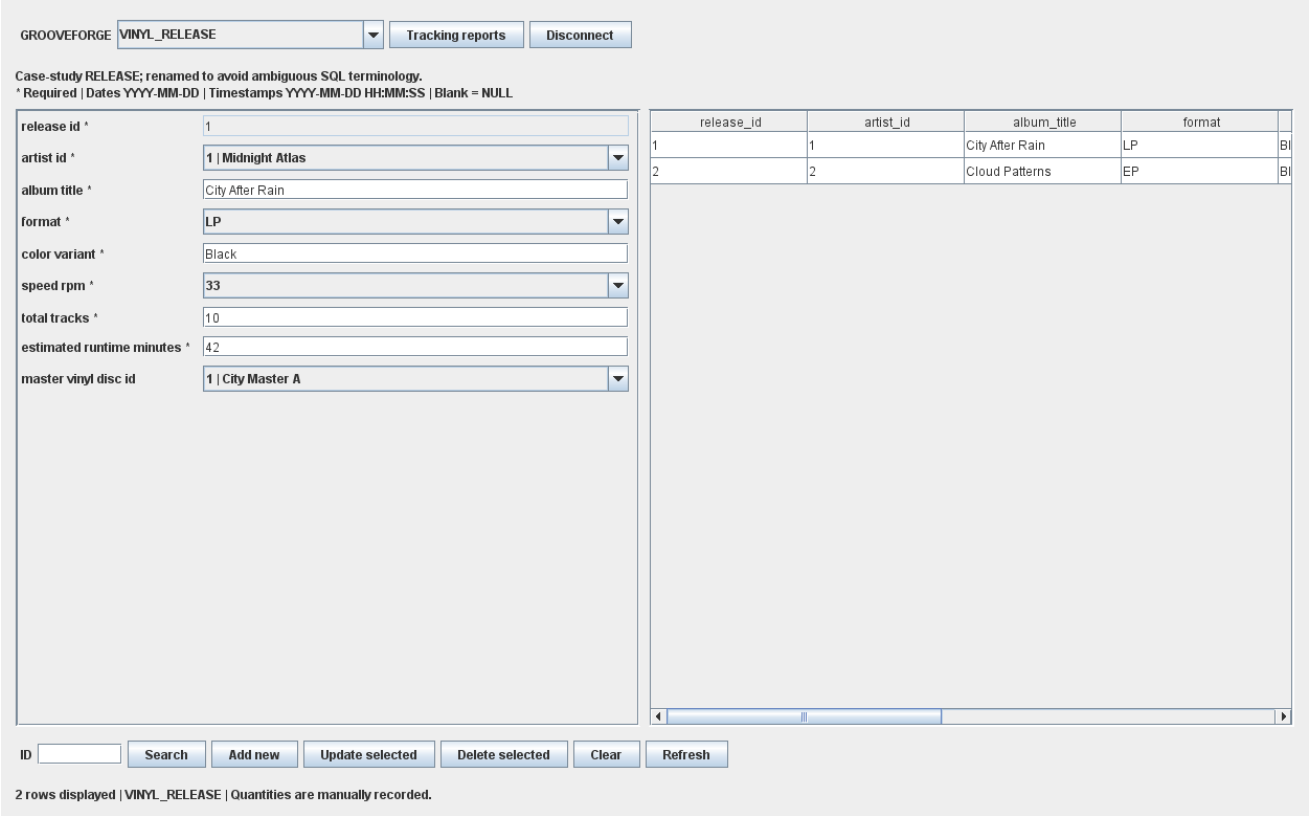
A selected row cannot accidentally be added as a duplicate: the user must Clear before Add. Updates require a selected or searched ID. Failed writes retain the form for correction. All resources use try-with-resources where appropriate.

Deletion and circular links

To delete a selected master disc, first clear its selection from VINYL_RELEASE, then remove the disc if no other references exist. To remove a release, remove or reassign its batches, orders, finished records and masters in dependency order. Do not disable constraints to make deletion succeed.

5. Demonstration: artist to selected master

Step	Input/action	Expected result
1	ARTIST -> Clear; enter Nova Test, Indie, Mira, 9000000099, mira@example.com, 2026-09-28; optional end date blank.	New artist ID generated.
2	VINYL_RELEASE -> choose new artist; album Demo Session; LP; Black; 33 rpm; 4 tracks; runtime 18; master blank.	New release ID generated.
3	MASTER_DISC -> choose that release; name Demo Master; created 2026-09-28; creator Riya Sen; Vault C; New.	New master ID generated.
4	VINYL_RELEASE -> select new release; choose Demo Master; Update.	Selected master saved.
5	Try selecting a master belonging to another release.	Oracle rejects the mismatched composite FK.

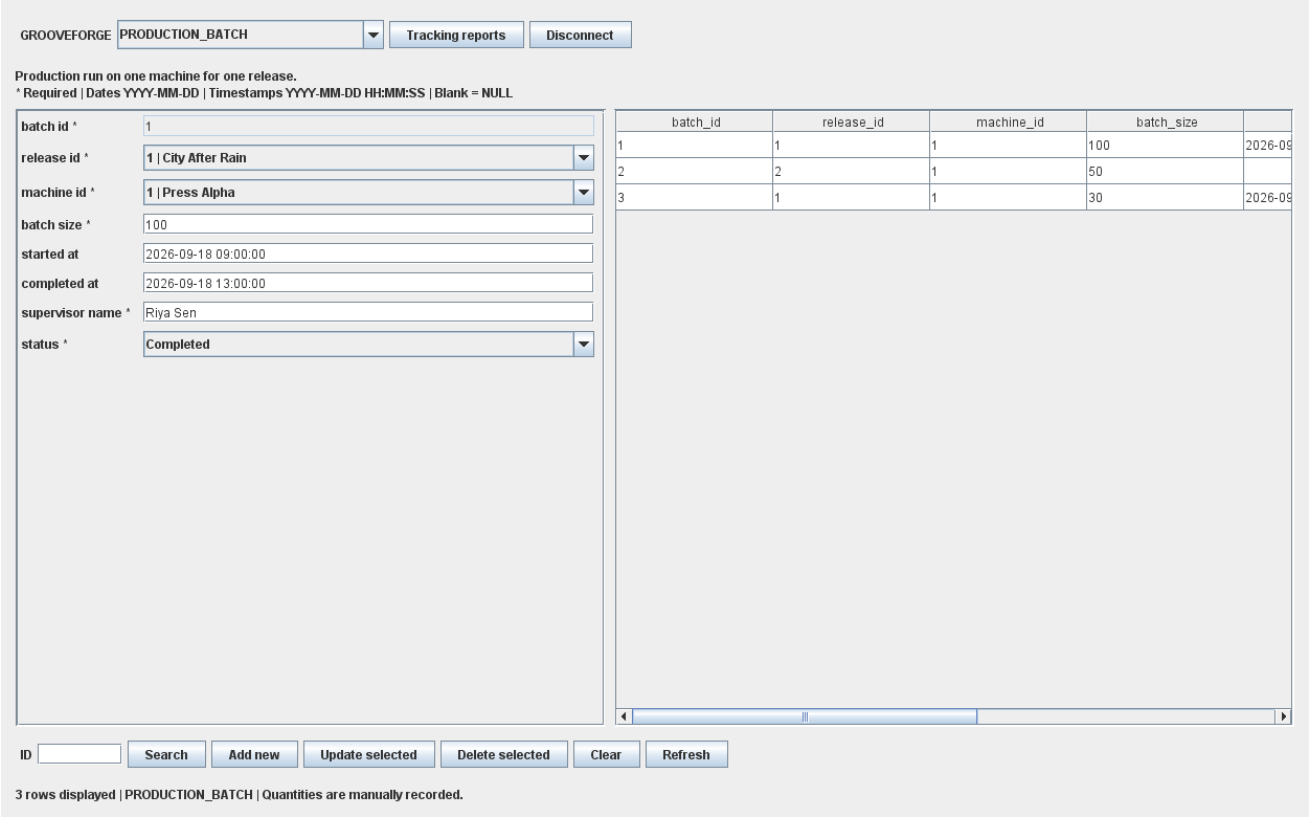


UI illustration: H2 fixture showing seeded release 1 and its selected master.

5.1 Demonstration: production and inventory tracking

Step	Operator action	Expected behavior
Machine	Review PRESSING_MACHINE status before creating a batch.	Operational suitability is an operator decision.
Batch	Create Scheduled batch for a release and a machine; enter batch_size and supervisor.	Batch saved; timestamps may be blank.
Start	Update status to In Progress and supply started_at.	Missing started_at is rejected.
Usage	Record batch/material/quantity/unit in BATCH_MATERIAL_USAGE.	Unit must match the material; stock is unchanged.
Quality	Add QUALITY_CHECK with sample_size, inspector, timestamp and result.	Inspection saved; batch status is unchanged.
Finish	Set batch Completed with completion timestamp; register serialized FINISHED_RECORD rows.	Batch/release mismatch or duplicate serial is rejected.
Shipping	Update CUSTOMER_ORDER to Shipped with shipped_date and tracking number.	Missing shipment details are rejected.

To demonstrate manual stock tracking, note material 1 quantity_in_stock = 800, add a restock event, and reload RAW_MATERIAL. It remains 800 until explicitly edited. This is correct for the supplied no-calculations scope.



UI illustration: H2 fixture; batch statuses are manually entered.

5.2 Tracking reports and expected outputs

These result rows were checked locally on the equivalent SQLite model and the report queries also ran on H2. They are expected to match the pristine Oracle seed data; formatting may differ. They are not claimed as actual Oracle outputs.

Release catalog

```
SELECT r.release_id, a.artist_name, r.album_title, r.format, r.color_variant FROM VINYL_RELEASE
    r JOIN ARTIST a ON a.artist_id = r.artist_id ORDER BY r.release_id;
```

release_id	artist_name	album_title	format	color_variant
1	Midnight Atlas	City After Rain	LP	Black
2	Monsoon Echo	Cloud Patterns	EP	Blue

Batch tracking

```
SELECT b.batch_id, r.album_title, m.machine_name, b.status, b.batch_size FROM PRODUCTION_BATCH b
     JOIN VINYL_RELEASE r ON r.release_id = b.release_id JOIN PRESSING_MACHINE m ON m.machine_id
     = b.machine_id ORDER BY b.batch_id;
```

batch_id	album_title	machine_name	status	batch_size
1	City After Rain	Press Alpha	Completed	100
2	Cloud Patterns	Press Alpha	Scheduled	50
3	City After Rain	Press Alpha	Quality Hold	30

5.3 Tracking reports and expected outputs

These result rows were checked locally on the equivalent SQLite model and the report queries also ran on H2. They are expected to match the pristine Oracle seed data; formatting may differ. They are not claimed as actual Oracle outputs.

Quality history

```
SELECT q.check_id, r.album_title, q.batch_id, q.overall_result, q.notes FROM QUALITY_CHECK q
    JOIN PRODUCTION_BATCH b ON b.batch_id = q.batch_id JOIN VINYL_RELEASE r ON r.release_id =
    b.release_id ORDER BY q.check_id;
```

check_id	album_title	batch_id	overall_result	notes
1	City After Rain	1	Pass	Sample approved
2	City After Rain	3	Fail	Held for review

Material usage

```
SELECT u.usage_id, u.batch_id, m.material_name, u.quantity_used, u.unit FROM
    BATCH_MATERIAL_USAGE u JOIN RAW_MATERIAL m ON m.material_id = u.material_id ORDER BY
    u.usage_id;
```

usage_id	batch_id	material_name	quantity_used	unit
1	1	Black Vinyl Pellets	18	Kilograms
2	1	Label Paper	200	Sheets
3	1	Outer Jacket Cardboard	100	Pieces

5.4 Tracking reports and expected outputs

These result rows were checked locally on the equivalent SQLite model and the report queries also ran on H2. They are expected to match the pristine Oracle seed data; formatting may differ. They are not claimed as actual Oracle outputs.

Orders awaiting dispatch

```
SELECT order_id, customer_name, release_id, status FROM CUSTOMER_ORDER WHERE status IN
    ('Pending','In Production','Quality Hold','Ready to Ship') ORDER BY order_id;
```

order_id	customer_name	release_id	status
1	Atlas Label	1	Ready to Ship
2	Echo Label	2	Pending

Record traceability

```
SELECT f.serial_number, r.album_title, b.batch_id, m.machine_name, f.current_location FROM
    FINISHED_RECORD f JOIN PRODUCTION_BATCH b ON b.batch_id = f.batch_id JOIN VINYL_RELEASE r ON
     r.release_id = f.release_id JOIN PRESSING_MACHINE m ON m.machine_id = b.machine_id ORDER BY
     f.record_id;
```

serial_number	album_title	batch_id	machine_name	current_location
GF-CAR-0001	City After Rain	1	Press Alpha	Warehouse Shelf A3
GF-CAR-0002	City After Rain	1	Press Alpha	Warehouse Shelf A3
GF-CAR-0101	City After Rain	3	Press Alpha	Quality Hold Area

5.5 Tracking reports and expected outputs

These result rows were checked locally on the equivalent SQLite model and the report queries also ran on H2. They are expected to match the pristine Oracle seed data; formatting may differ. They are not claimed as actual Oracle outputs.

Maintenance follow-up

```
SELECT m.machine_name, l.maintenance_date, l.issue_reported, l.next_due_date FROM
    MAINTENANCE_LOG l JOIN PRESSING_MACHINE m ON m.machine_id = l.machine_id ORDER BY l.log_id;
```

machine_name	maintenance_date	issue_reported	next_due_date
Press Alpha	2026-09-01	Routine service	2026-10-01
Press Beta	2026-09-20	Hydraulic leak	2026-09-30

Selected master discs

```
SELECT r.release_id, r.album_title, d.disc_name, d.disc_condition FROM VINYL_RELEASE r LEFT JOIN
     MASTER_DISC d ON d.disc_id = r.master_vinyl_disc_id AND d.release_id = r.release_id ORDER
    BY r.release_id;
```

release_id	album_title	disc_name	disc_condition
1	City After Rain	City Master A	Good
2	Cloud Patterns	Cloud Master A	New

5.6 Tracking reports and expected outputs

These result rows were checked locally on the equivalent SQLite model and the report queries also ran on H2. They are expected to match the pristine Oracle seed data; formatting may differ. They are not claimed as actual Oracle outputs.

Releases without batches

```
SELECT r.release_id, r.album_title FROM VINYL_RELEASE r WHERE NOT EXISTS (SELECT 1 FROM
    PRODUCTION_BATCH b WHERE b.release_id = r.release_id) ORDER BY r.release_id;
```

Expected result: no rows. Both seeded releases already have batches.

Failed quality checks

```
SELECT check_id, batch_id, visual_defects_found, audio_defects_found, notes FROM QUALITY_CHECK
    WHERE overall_result = 'Fail' ORDER BY check_id;
```

check_id	batch_id	visual_defects_found	audio_defects_found	notes
2	3	Warping	None	Held for review

6. Testing and evidence status

Verification	Status
Java source compilation	PASS: Eclipse Java compiler 3.26, Java 8 language target; no compile errors or warnings after fixes.
Portable relational model	PASS: 14 positive/negative checks on SQLite-equivalent schema.
SQL/JDBC portable fixture	PASS: original Oracle-style table/seed SQL executed on H2 Oracle mode after SQL*Plus directives were removed.
Swing modules	PASS: all 14 forms loaded; add/search/update/delete executed through App methods using the H2 fixture.
Reports	PASS: all 10 report queries executed on both portable fixtures.
Oracle database / login / PL/SQL suite	NOT RUN HERE: requires the lab Oracle instance and compatible JDBC driver.
Manual output	PDF rendered and visually reviewed; source files and diagrams included.

Negative acceptance checks supplied for Oracle

Case	Expected result
Missing artist parent	rejected
Cross-release selected master	rejected
Finished record mismatched release	rejected
Usage unit mismatch	rejected
Negative raw stock	rejected
Duplicate serial	rejected
Invalid machine status	rejected
Bad contract chronology	rejected
Completion without start	rejected
Shipment details missing	rejected
Warehouse over capacity	rejected
Delete referenced machine	rejected
Insert/search/update/delete on independent row	PASS
Restock leaves manual stock unchanged	PASS

Run sql/05_acceptance_tests.sql in the lab before altering seed rows. It uses savepoints, checks Oracle error codes, and fails the run if an invalid write is accepted. Its output is captured as acceptance_output.txt.

6.1 Lab UI test matrix and evidence checklist

Test	Expected outcome
Every module opens	Rows and parent choices load without an error.
Add valid row in each module	Generated ID and committed row appear after refresh.
Search existing / missing ID	Form fills / ID not found.
Update selected row	Edited value persists after disconnect/reconnect.
Delete independent test row	Row disappears and remains absent after reconnect.
Required field blank	Validation message; no write.
Malformed date or excessive precision	Input rejected before JDBC write.
Duplicate serial number	Unique constraint error.
Referenced parent deletion	Foreign-key error; parent remains.
Invalid release/master or batch/release	Composite foreign-key error.
Rollback demonstration	Temporary status changes disappear.

Capture these actual lab outputs

1. JDK/compiler versions.
2. Oracle instance/version and connected GROOVEFORGE user.
3. Tables, sequences and seed row counts.
4. Acceptance script output.
5. Valid login/dashboard and rejected login.
6. Add/search/update/delete in all 14 modules.
7. Matching and mismatching selected-master examples.
8. At least three report results.
9. Transaction rollback before/after.
10. A disconnect/reconnect persistence check.

Do not relabel the portable fixture screenshots as Oracle screenshots. Replace or supplement them with your actual laboratory captures for submission. No student name, roll number, guide signature or institutional approval has been fabricated.

6.2 Troubleshooting

Symptom	Cause / action
ORA-00955	Object already exists. Inspect the schema; use a fresh schema for a clean build. Oracle DDL is not fully rolled back.
ORA-00942	Wrong user or tables missing. Confirm SELECT USER and run the schema build.
ORA-01017	Incorrect schema credentials; verify in SQL*Plus.
ORA-12541 / connection refused	Listener/host/port problem; check the lab connection details.
ORA-12514 / ORA-12505	Wrong service or SID. Match the JDBC URL to the installed database.
ORA-65096	Local user attempted in CDB root; connect to the lab PDB.
ORA-02291	Parent missing or composite linkage inconsistent. Check the selected IDs and release/unit pair.
ORA-02292	Child rows still reference the parent. Keep the parent or handle the children first.
ORA-02290	CHECK rule violated: inspect dates, quantities and status requirements.
ORA-00001	Duplicate serial, warehouse address or key.
ClassNotFoundException	Oracle JDBC JAR is absent from lib or NetBeans Libraries.
No javac / unsupported class version	JDK missing or build/run JDK mismatch. Choose a consistent lab runtime.
Rows visible in SQL*Plus but not GUI	COMMIT changes in the other session and Refresh.
Selected master cannot be deleted	Clear the release master selection first.
Restock does not increase stock	Expected: stock is a manually maintained snapshot.

7. Viva questions with model answers

Why use a separate schema?

It separates project objects and privileges from administrator/system objects.

Why use sequences?

They generate concurrent-safe numeric identifiers without requiring modern identity columns. Gaps are normal.

Why is the master link nullable?

It breaks the insertion cycle: create release, create its master, then select the master.

Why use a composite master FK?

A simple disc FK proves existence only. The pair also proves the disc belongs to the same release.

Why retain release_id on finished records?

The supplied model requires it. A composite batch/release FK prevents contradictory values.

Is the schema fully normalized to 3NF?

No blanket claim is made. Repeated unit and release fields are controlled source-required redundancies.

Why does stock not change on receipt?

The case study specifies tracking without calculations; stock and event quantities are manually recorded.

What does PreparedStatement prevent?

Bound values are treated as data, reducing SQL-injection risk. Identifiers come from a fixed allowlist.

7.1 Viva questions - application and testing

Why commit?

It makes successful changes durable and visible to other sessions.

Why roll back?

It removes uncommitted changes after a failed operation or canceled demonstration.

What does a foreign-key delete error mean?

A child still references the parent. Rejecting the delete preserves integrity.

Why are employees/locations standalone?

The case study supplies text names/locations but no foreign-key attributes connecting those registers.

What does the employee Admin role grant?

Nothing automatically. It is a descriptive field; database authentication controls this client.

What is a ResultSet?

The cursor-like JDBC result of a query, read to populate the Swing table.

Why SwingWorker?

It performs JDBC work in the background so the UI event thread stays responsive.

What remains to validate on Oracle?

Database-specific DDL/PLSQL behavior, JDBC compatibility, actual authentication and end-to-end lab CRUD.

8. Package map, submission and references

Path	Use
sql/00_create_user.sql	One-time administrator setup.
sql/build.sql	Calls schema, seed and verification scripts in order.
sql/01_schema.sql	All 14 tables, composite constraints, sequences and indexes.
sql/02_sample_data.sql	Complete fictional demonstration data.
sql/03_reports.sql	Ten tracking queries.
sql/04_verify.sql	Tables, sequences, row counts, constraint state.
sql/05_acceptance_tests.sql	Oracle positive/negative checks with rollback.
sql/06_transaction_demo.sql	Explicit savepoint/rollback example.
src/grooveforge/	Complete App, Model and TestConnection Java sources.
diagrams/index.html	Openable SVG diagram atlas, all 101 attributes.
evidence/	Portable checks, compile/test status and labeled UI images.
README.md	Quick start and scope decisions.

Sources used

1. User-supplied Case Study Smart Vinyl Record Press.txt: authoritative domain entities and attributes.
2. User-supplied Final-Hospital_Management_System_Detailed_Manual_With_Outputs.pdf, 37 pages: reference structure, Java Swing/JDBC/Oracle workflow and output-checkpoint style.
3. Oracle JDBC Developer Guide, Getting Started: driver, database and JDK compatibility. Official documentation.
4. Oracle SQL Language Reference, constraint: composite foreign-key requirements. Official documentation.

Submission preparation: enter the actual student/team details in the institution-required cover format, run the Oracle lab checks, attach actual outputs, and explain the documented design choices. This manual does not assert institutional approval or a grade.